

I started with how agents would use a Slack workspace



Slack workflows



Automatic result checks

- I studied how agents observe and act on pages, then reviewed Slack mockups and agent environments.
- I mapped search, threads, and message edits to agent actions and saved changes I could verify.
- I built the app, action loop, and result checks separately so I could test each part.



Research
computer use



Map Slack
workflows



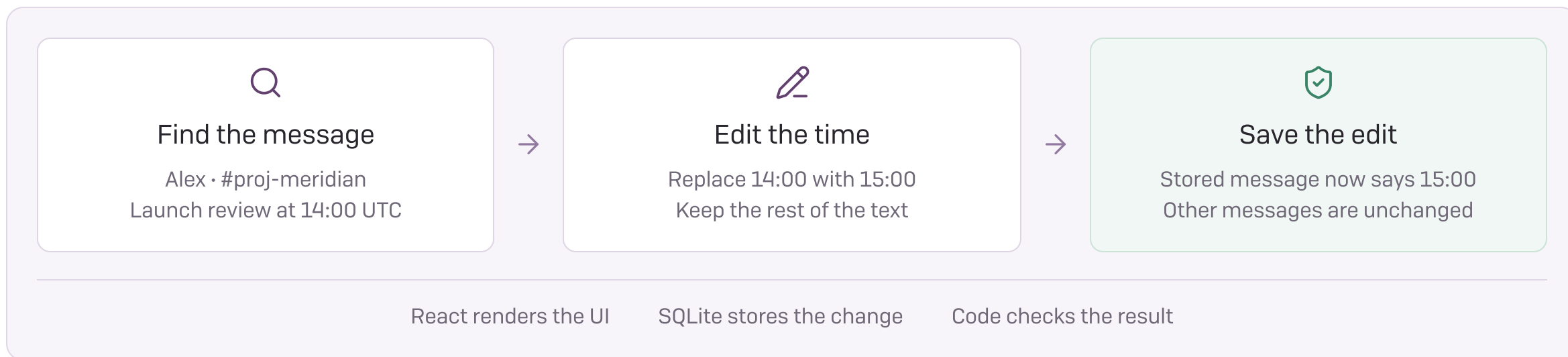
Build the
environment



Test and
record runs

I started with a task that needs a **real saved change**

- The agent must find Alex's launch-review message and change its time from 14:00 to 15:00 UTC.
- Typing the new time is not enough: it must save the edit without changing other messages.



- A static mockup cannot test persistence. A full Slack backend adds services these tasks do not need.
- Fictional data · Task abbreviated here · [Try the workspace without a key](#) · [Prior-work review](#).

The task set covers 18 workflows

Six focused tasks

Update a topic

Channel → edit topic → save

Reply in a thread

Find request → open thread → reply

Edit a message

Find own message → edit → save

Triage an incident

Find active incident → react → pin

Delete a draft

Find own draft → delete → confirm

Send a handoff

Read current details → compose DM

Coordinate work

Coordinate a release *

Read sources → edit → reply → topic → DM

Close out an incident

Read resolution → replace pin → reply → topic

Refresh saved work

Find current sources → replace saved items → DM

Record a design decision

Read approval → react → pin → save → details

Repair an outdated handoff

Read correction → edit DM → react → save

Complete a QA sign-off

Read test results → reply → fix reaction → save

Combine several changes

Publish a release update

Read source → format update → pin → delete draft

Brief the on-call engineer

Read rotation → DM primary → save → details

Correct a threaded estimate

Read review → edit own reply → react → reply

Refresh the launch board

Find plan → replace pin → save → topic

Hand off an approved design *

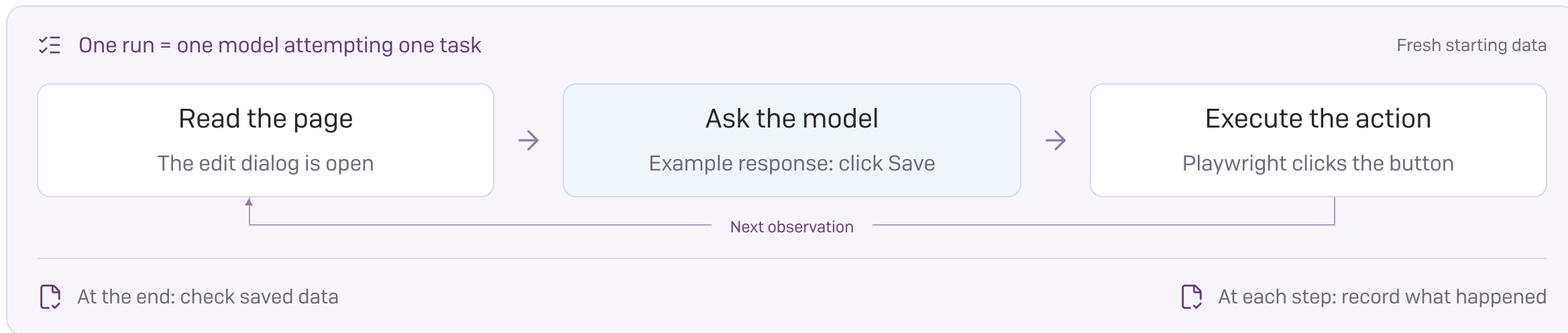
Read decision → react → DM → reply → save

Prepare a release retrospective

Read three sources → summarize → pin → save → DM

- These diagrams summarize the intended workflows. Exact instructions and preservation checks remain in each task.
- * Two original tasks contain wording problems. Their results remain unchanged and are discussed later.

The model proposes an action; the harness executes it



- The harness accepts only supported actions, so a model response cannot execute arbitrary code.
 - After Save, it reads the updated page so the model can check whether the edit appeared.
 - Action, time, and spending limits stop a stuck agent from making model calls indefinitely.
-
- The example shows browser control. The API path executes application operations instead.

The model can use **four interfaces** to the same app

Interface	Observation → action	Interaction
 Pixels	Screenshot → mouse and keyboard	Browser
 Accessibility	Named page controls → click or fill	Browser
 Page JSON	Structured page controls → click or fill	Browser
 Actor API	Workspace data → direct changes	Tool use

- A browser edit requires finding the editor, entering text, and saving the change.
- The actor API can request that edit directly, so the workspace changes without a browser click.
- The same grader checks the result, but the model uses different information and actions to reach it.

-
- Page JSON describes page controls, not the underlying database. Actor API access exposes broader structured data.
 - The harness uses a shared JSON action protocol, not each provider's native computer-use protocol.

Success means the requested change was saved

- A model can type the right text and click Cancel, so the screen alone cannot prove success.
- The grader is code that checks the stored messages, topics, and other workspace data.

 Required saved message “Launch review is at 15:00 UTC. Please bring the final checklist.”		
 Right change Exact text in Alex's original message	 No extra changes No new message or unrelated edit	Both pass Reward = 1

- The grader checks the final state. It does not penalize a wrong action that the agent fully undoes.
- Task definitions and result checks.

I tested correct and nearly correct outcomes separately

 CAN THE UI DO IT?

Run a scripted solution

Search → edit → Save → inspect the stored message

The requested change must persist.

 CAN THE GRADER CATCH A NEAR MISS?

Alter a known correct result

Wrong destination, wrong author, or one extra change

The altered result must fail.

 CAN I EXPLAIN A MODEL'S RESULT?

Reopen the recorded run

Read its actions and check its saved data again

The recorded grade must agree.

- A failed run can come from the app, the grader, or the model, so I check each separately.
- I inspect the saved actions and data to explain the failure without buying another attempt.
- A scripted solution tests my intended answer, not whether another reader understands the task.

-
- [Software verification](#) · The longer-workflow grader tests cover 84 valid and 2,583 invalid states across seven seeds.

Each model starts with its own copy of the task

- If a previous run leaves the message at 15:00, the next model starts with work already done.
- Each run gets its own SQLite database and browser session, so its actions change only its own workspace.

Same task and starting data • separate copies

RUN A

Browser session A → database A

Agent saves its edit

Launch review at 15:00

RUN B

Browser session B → database B

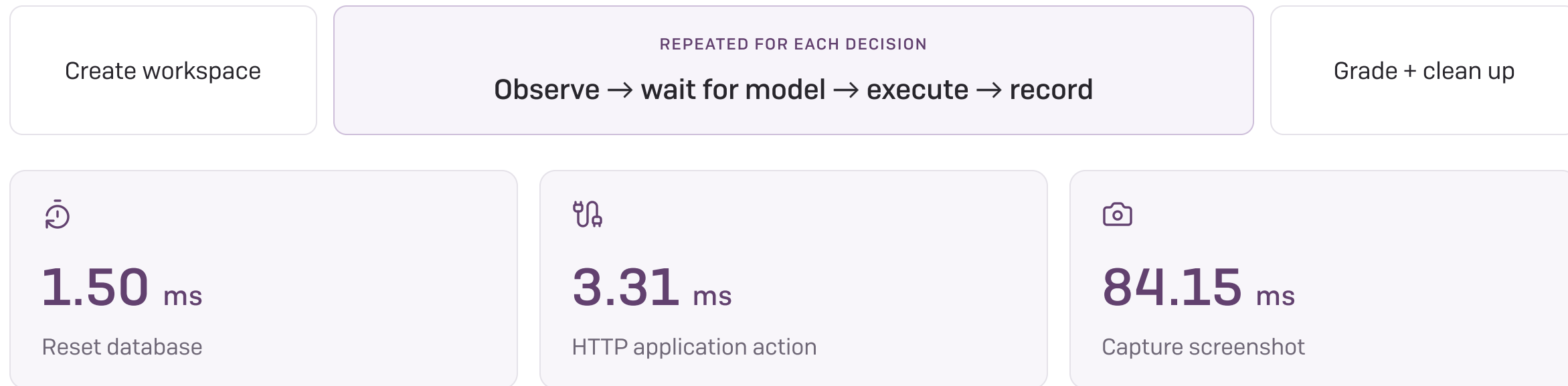
No action in this run

Launch review stays at 14:00

In the live demo, starting the next model restores the task's starting data, not the previous model's edits.

- Tested: write to A, check B is unchanged; close A, check B still works. Cost: separate files and browser setup.
- The actor and control servers share a process. Separate data is not isolation from host crashes or malicious code.

Every extra decision adds **another model request**



- Opening the editor, entering text, and saving can each require a model decision.
- In the live viewer, the workspace waits during the model request and updates after the action executes.

-
- A faster reset helps between runs, not between decisions. Local M5 Max / Node 24 medians exclude model calls and are not a full-run estimate.
 - Samples and setup: 100 stored sessions; screenshots from 10 sequential browser contexts, not 100 concurrent agents.

Browsers and recordings have **separate resource costs**

WORKSPACE DATA

One SQLite file per run

The next model does not inherit earlier edits

Measured: 48 KiB per fixture after one post

BROWSER EXECUTION

Fresh browser per experiment run

No browser state carried into the next run

Cost: startup time and browser memory

EVIDENCE

Record actions and workspace states

Review failures without new model calls

Cost: growing files and disk writes

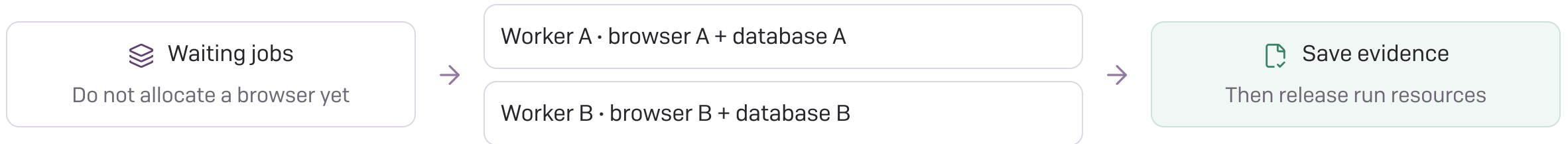
- Stored actions and states let the replay show each run without calling the model again.
- The 306-run collection paused when free disk space fell below its storage safeguard.

-
- The storage test did not measure full browser memory. Current runs create and close their own environment; browser reuse is untested.
 - API execution can omit the browser when visual recording is disabled. The reported interface study kept observer recording enabled.
 - Recorded storage stop · Existing evidence was preserved.

A queue would limit how many browsers run at once

- Starting a browser for every visitor could exhaust memory and slow or interrupt active runs.
- I would queue new runs until a worker is free, while keeping each active run's data separate.

PROPOSED, NOT IMPLEMENTED OR LOAD-TESTED



A visitor would wait for a free worker before their run starts. More workers could reduce the queue, not model response time.

- The collections ran sequentially, so capacity is unproven. Test 1, 4, 8, and 16 workers for throughput, memory, provider limits, and crash cleanup.
- Use separate processes or containers for stronger containment. Browser pooling trades lower startup cost for shared failure risk.

The reusable harness made **model comparisons possible**

- Once the harness worked, I could change the model without rebuilding the test.
- I expanded the assignment into a benchmark prototype with two separate comparisons.
- The first compares models using accessibility controls, and the second compares four interfaces.

Original assignment



Working environment

Focused Slack workflows
Known starting state



Repeatable harness

Reset · act · check · record
Select a model



Expanded scope



Benchmark prototype

306 model-comparison runs
96 interface-comparison runs

The next slides cover the test conditions, results, and limitations of each comparison.

Comparison 1: 17 models, using accessibility controls

$$\begin{array}{ccccccc} \text{📁} & & \text{☑️} & & \text{🛡️} & & \\ 17 & \times & 18 & \times & 1 & = & 306 \\ \text{model routes} & & \text{tasks} & & \text{attempt per pair} & & \text{recorded attempts} \end{array}$$

- Each model receives named page controls and returns one JSON action per request.
- All 306 runs use the same starting data, recent-four history, and no site guide.
- This tests model performance under one interface, not the best interface for each model.

40 actions 180 s per run \$5 estimated allowance per run

- Seed 1042 • 4,096 output tokens per request • identical graders • provider-default reasoning.
- I kept 105 earlier runs and added 201 new runs without replacing failures.
- [Node 24 setup and API key instructions](#) • [Exact routes, limits, and run history](#). Provider routing and collection time were not controlled.

Results for 17 models

1 per task/model · Collection complete

306 / 306 attempted · 152 passed · 61 incomplete · 93 blocked

\$189.3110

Usage estimate

\$3.7794

Unresolved · 35 calls

\$193.0904

Total allowance · 306 trials

Model ↕	Trials ↕	Passed ↕	Incomplete ↕	Blocked ↕	Median time ↕	Usage estimate ↕	Unresolved ↕	Total allowance ↕
 GPT-6 Astra	18 / 18	14 / 18	1	3	27.1s	\$35.9623	\$0.0000	\$35.9623
 GPT-6.1 Sol	18 / 18	14 / 18	4	0	28.5s	\$8.4352	\$0.0000	\$8.4352
 GPT-6 Luna	18 / 18	9 / 18	8	1	39.4s	\$0.5394	\$0.0092	\$0.5486
 GPT-OSS 120B	18 / 18	5 / 18	2	11	180.1s	\$0.6682	\$0.1358	\$0.8040
 Claude Fable 5.1	18 / 18	13 / 18	1	4	62.8s	\$48.4714	\$0.0000	\$48.4714
 Claude Opus 5.5	18 / 18	13 / 18	4	1	38.3s	\$28.6987	\$0.3741	\$29.0728
 Claude Sonnet 5.5	18 / 18	7 / 18	4	7	39.9s	\$13.9323	\$0.3304	\$14.2627
 Claude Haiku 4.5	18 / 18	6 / 18	3	9	28.3s	\$3.7921	\$0.0000	\$3.7921
 Grok 4.7	18 / 18	10 / 18	6	2	34.3s	\$11.9101	\$0.3809	\$12.2910
 Qwen 3.8 Max	18 / 18	7 / 18	1	10	125.1s	\$6.9999	\$1.0639	\$8.0638
 DeepSeek V4 Pro	18 / 18	8 / 18	1	9	106.8s	\$4.6816	\$0.2547	\$4.9363
 DeepSeek V4.1 Flash	18 / 18	7 / 18	3	8	65.9s	\$1.3825	\$0.0541	\$1.4365
 GLM 5.3	18 / 18	7 / 18	0	11	125.3s	\$3.6307	\$0.3453	\$3.9760
 GLM 5.3 Flash	18 / 18	8 / 18	0	10	46.2s	\$0.4444	\$0.0000	\$0.4444
 Kimi K3	18 / 18	11 / 18	3	4	50.0s	\$14.0051	\$0.7958	\$14.8009
 MiniMax M3	18 / 18	5 / 18	10	3	25.9s	\$1.4698	\$0.0353	\$1.5052
 Nemotron 3 Ultra	18 / 18	8 / 18	10	0	50.1s	\$4.2874	\$0.0000	\$4.2874

Passed Incomplete Blocked Not run

- 18 public tasks. 1 run per task and model. All use accessibility controls. Median time includes blocked runs.
- Total allowance adds the usage estimate to money reserved for missing usage. These are estimates, not invoices. Including earlier tests: \$194.1058 of the \$300 budget. [Full costs CSV](#) · [JSON](#)
- 105 earlier runs are included. No retries. One run per task cannot establish a reliable model ranking. Two tasks have unclear instructions. [Read the failure analysis.](#)
- [Test plan](#) · [Recorded runs](#) · [Download all trials \(CSV\)](#)

A correct grader can still test an unclear instruction

- One task asks the agent to send the approved design to a teammate.
- The workspace names the design Willow, but my task quotes a message containing DESIGN without explaining the substitution.

INSTRUCTION EXCERPT	DM its named handoff recipient “Implement DESIGN navigation; accessibility approved.”
SIX COMPLETED RUNS SENT	“Implement DESIGN navigation; accessibility approved.”
THE GRADER REQUIRED	“Implement Willow navigation; accessibility approved.”

Missing check: could an independent reader get the same answer from this instruction?

- This explains a problem in six traces, not all 17 failures. A second task had an unclear source location.
- Recorded actions and failure analysis. Keep the recorded grades. Fix the instruction in a new task version before collecting new results.

What the early interface tests taught me

- These runs showed the harness could record successes, wrong actions, and interrupted runs.
- They used different task and model combinations, so their pooled scores could not compare interfaces.
- For the next study, I gave every model the same six tasks through all four interfaces.



Later study: one model + one task

Four fresh workspaces • identical starting data

Accessibility

Page JSON

Pixels

API • tool use

Same final-state checks • 4 models × 6 tasks × 4 interfaces = 96 attempts

- Early development traces are separate evidence, not a third benchmark or part of the later scores.
- Later study: randomized interface order • fixed seed, history, reasoning, and limits • \$25 total • no retries. Full design and controls.

Interface comparison: 96 recorded attempts

Passed / attempted · median elapsed time in seconds · six tasks per cell

Model ↕	Accessibility ↕	Page JSON ↕	Pixels ↕	Actor API ↕	Allowance ↕
 GPT-6.1 Sol	<u>6 / 6</u> 0 incomplete · 0 blocked All 28.4 s Passed 28.4 s	<u>5 / 6</u> 0 incomplete · 1 blocked All 33.2 s Passed 50.3 s	<u>1 / 6</u> 0 incomplete · 5 blocked All 107.4 s Passed 76.7 s	<u>6 / 6</u> 0 incomplete · 0 blocked All 10.0 s Passed 10.0 s	\$3.9430
 Claude Sonnet 5.5	<u>4 / 6</u> 0 incomplete · 2 blocked All 26.3 s Passed 10.2 s	<u>4 / 6</u> 1 incomplete · 1 blocked All 26.5 s Passed 10.8 s	<u>2 / 6</u> 1 incomplete · 3 blocked All 23.7 s Passed 19.2 s	<u>5 / 6</u> 1 incomplete · 0 blocked All 9.9 s Passed 7.8 s	\$7.2784
 Qwen 3.8 Max	<u>5 / 6</u> 1 incomplete · 0 blocked All 26.7 s Passed 14.7 s	<u>4 / 6</u> 1 incomplete · 1 blocked All 23.9 s Passed 36.6 s	<u>0 / 6</u> 0 incomplete · 6 blocked All 55.2 s Passed N/A	<u>5 / 6</u> 0 incomplete · 1 blocked All 11.9 s Passed 6.3 s	\$5.4829
 Grok 4.7	<u>3 / 6</u> 1 incomplete · 2 blocked All 20.8 s Passed 7.8 s	<u>3 / 6</u> 3 incomplete · 0 blocked All 17.2 s Passed 7.8 s	<u>0 / 6</u> 1 incomplete · 5 blocked All 40.5 s Passed N/A	<u>5 / 6</u> 1 incomplete · 0 blocked All 9.0 s Passed 5.2 s	\$6.0573

[All traces and replays ↗](#) [Costs CSV ↓](#) [Paired results and timing ↗](#)

- All-attempt medians include early stops, while passed-run medians use only successful tasks.
- Passed tasks differ across cells, so these medians do not establish a speed ranking.

- Verified: 58 passed · 11 incomplete · 27 blocked. Usage estimate \$20.7180 + unresolved \$2.0435 = \$22.7616 of \$25.
- N/A means no passed task. Time includes model calls and browser work, not just model latency. Click a pass count for traces and replays.

The API result does not show better computer use

EXAMPLE: API TOPIC CHANGE



Set the topic directly

One application operation
No dialog to navigate

SAME CHANGE IN THE BROWSER



Open → fill → Save

Several UI actions
Each requires another decision

PIXEL RESULTS



19 / 24 stopped early

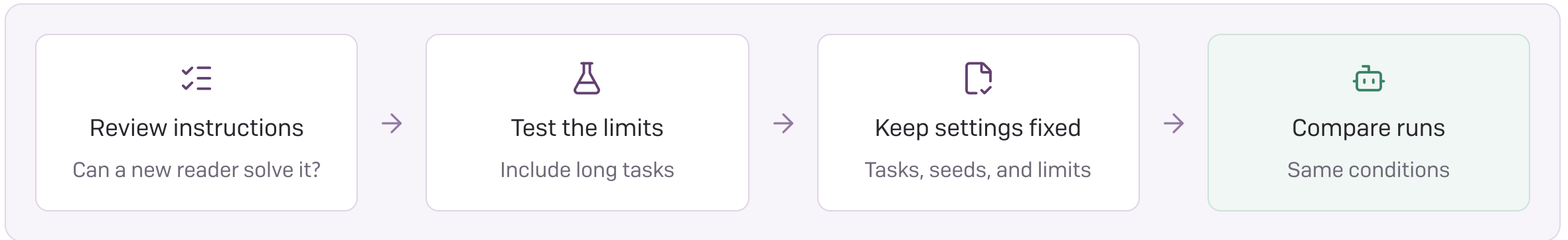
14 connection failures
3 output limits • 2 timeouts

- The same final check does not make the routes equally difficult: API access skips browser interaction.
- A pixel failure can reflect image delivery, keyboard behavior, or a service interruption, not just a wrong decision.
- One attempt per condition cannot establish a reliable winner.

-
- 17 shared passes for API/accessibility. 24 API/accessibility pairs: 17 both passed • 4 API only • 1 accessibility only • 2 neither passed. 5 pairs include a blocked attempt.
 - On 17 shared passes, median accessibility-minus-API differences were +2 actions, +5.462 seconds, and +\$0.13008. These are selected successful pairs, not a general speed estimate.
 - Pixels used low-detail images on macOS. Time includes shared-host activity. [Definitions, costs, and all matched pairs.](#)

The next version needs validated tasks and repeated runs

- Ask someone outside the project to solve each task from its instructions.
- Compare their interpretation with the expected result.
- Set run limits with both short and long tasks.
- Use new task variants and repeat runs to measure reliability.



The current results describe these attempts. Repeated runs are needed to estimate reliability.

- [Detailed engineering review](#) · [Remaining validation work](#). These are next steps, not completed results.

Demo and design questions



01 · TRY

Use the workspace

Search, open a thread, edit



02 · COMPARE

Choose a model and task

All 306 attempts and costs



03 · REVIEW

Inspect the run

Actions, changes, result checks

- Which workflows should the next version test?
- Should the score include wrong actions that the agent later undoes?
- How many agents must run at once, and what should each run cost?

-
- Relay can reset, execute actions, and return a reward. No reinforcement learning training has been run.
 - [Code and setup](#) · [Requirements and remaining work](#).